

COMUNICAÇÃO BASEADA EM RECURSOS: REST E HTTP

Objetivos de aprendizagem

- Explicar REST como estilo arquitetural e HTTP como protocolo de aplicação.
- Projetar recursos e URIs coerentes.
- Usar corretamente métodos e códigos de status HTTP.
- Implementar, testar e evoluir uma API REST com FastAPI.
- Analisar idempotência, concorrência, erros e observabilidade.

4.1 REST não é sinônimo de “JSON sobre HTTP”

REST (Representational State Transfer) é um estilo arquitetural para sistemas hipermídia distribuídos. Na prática de APIs, seus princípios aparecem em recursos endereçáveis, interface uniforme, mensagens autocontidas e ausência de sessão de aplicação mantida pelo servidor entre requisições. HTTP fornece métodos, semântica, cabeçalhos, cache e códigos de status; JSON é apenas uma representação possível.

4.2 Recursos e URIs

Recursos representam entidades ou coleções relevantes do domínio. Prefira substantivos e relações estáveis: /dispositivos, /dispositivos/42, /dispositivos/42/leituras. Evite modelar cada ação como verbo no caminho quando a semântica pode ser expressa pela combinação de recurso e método HTTP.

Tabela 1 - Semântica dos métodos HTTP

Método	Semântica típica	Seguro?	Idempotente?	Exemplo
GET	Obter representação	Sim	Sim	GET /dispositivos/42
POST	Criar ou processar comando	Não	Não por definição	POST /leituras
PUT	Substituir representação no URI conhecido	Não	Sim	PUT /dispositivos/42
PATCH	Atualizar parcialmente	Não	Depende da operação	PATCH /dispositivos/42
DELETE	Remover recurso	Não	Sim quanto ao efeito final	DELETE /dispositivos/42

4.3 Códigos de status

- 200 OK: operação concluída com corpo de resposta.
- 201 Created: novo recurso criado; idealmente informar sua localização.
- 204 No Content: sucesso sem corpo.
- 400 Bad Request: entrada inválida em nível geral.
- 401 Unauthorized: autenticação ausente ou inválida.
- 403 Forbidden: autenticado, porém sem autorização.
- 404 Not Found: recurso inexistente.
- 409 Conflict: conflito com o estado atual.
- 422 Unprocessable Content: representação sintaticamente válida, mas semanticamente inválida em frameworks que adotam esse código.
- 500 Internal Server Error: erro inesperado no servidor.
- 503 Service Unavailable: serviço temporariamente indisponível.

4.4 Laboratório: API de dispositivos

O domínio de laboratório será reutilizado em vários capítulos: dispositivos enviam ou possuem leituras, e diferentes componentes consomem essas informações. Primeiro criaremos uma API REST em memória; depois o estudante poderá substituir a persistência por banco de dados.

Ambiente

- Python 3.10 ou superior.
- Ambiente virtual recomendado.
- FastAPI e servidor ASGI.
- Biblioteca requests para o cliente.

Terminal

```
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell
# .venv\Scripts\Activate.ps1

pip install "fastapi[standard]" requests
```

Implementação passo a passo

Começamos com modelos de dados. O servidor mantém um dicionário em memória para tornar visível a semântica HTTP sem introduzir ainda uma camada de banco. Pydantic valida as representações de entrada.

main.py

```
from datetime import datetime, timezone
from typing import Dict, List
from fastapi import FastAPI, HTTPException, Response, status
from pydantic import BaseModel, Field

app = FastAPI(title="API de Dispositivos", version="1.0")

class DispositivoEntrada(BaseModel):
    nome: str = Field(min_length=2, max_length=80)
    local: str = Field(min_length=2, max_length=120)

class Dispositivo(DispositivoEntrada):
    id: int

class LeituraEntrada(BaseModel):
    valor: float
    unidade: str = Field(min_length=1, max_length=16)

class Leitura(LeituraEntrada):
    dispositivo_id: int
    instante: str

dispositivos: Dict[int, Dispositivo] = {}
leituras: List[Leitura] = []
proximo_id = 1

@app.get("/dispositivos", response_model=list[Dispositivo])
def listar_dispositivos():
    return list(dispositivos.values())

@app.post("/dispositivos", response_model=Dispositivo, status_code=status.HTTP_201_CREATED)
def criar_dispositivo(entrada: DispositivoEntrada, response: Response):
    global proximo_id
    d = Dispositivo(id=proximo_id, **entrada.model_dump())
    dispositivos[d.id] = d
    proximo_id += 1
    response.headers["Location"] = f"/dispositivos/{d.id}"
    return d
```

```

@app.get("/dispositivos/{dispositivo_id}", response_model=Dispositivo)
def obter_dispositivo(dispositivo_id: int):
    d = dispositivos.get(dispositivo_id)
    if not d:
        raise HTTPException(status_code=404, detail="Dispositivo não encontrado")
    return d

@app.put("/dispositivos/{dispositivo_id}", response_model=Dispositivo)
def substituir_dispositivo(dispositivo_id: int, entrada: DispositivoEntrada):
    if dispositivo_id not in dispositivos:
        raise HTTPException(status_code=404, detail="Dispositivo não encontrado")
    d = Dispositivo(id=dispositivo_id, **entrada.model_dump())
    dispositivos[dispositivo_id] = d
    return d

@app.delete("/dispositivos/{dispositivo_id}", status_code=status.HTTP_204_NO_CONTENT)
def remover_dispositivo(dispositivo_id: int):
    if dispositivo_id not in dispositivos:
        raise HTTPException(status_code=404, detail="Dispositivo não encontrado")
    del dispositivos[dispositivo_id]
    return Response(status_code=status.HTTP_204_NO_CONTENT)

@app.post("/dispositivos/{dispositivo_id}/leituras", response_model=Leitura, status_code=201)
def registrar_leitura(dispositivo_id: int, entrada: LeituraEntrada):
    if dispositivo_id not in dispositivos:
        raise HTTPException(status_code=404, detail="Dispositivo não encontrado")
    leitura = Leitura(
        dispositivo_id=dispositivo_id,
        instante=datetime.now(timezone.utc).isoformat(),
        **entrada.model_dump(),
    )
    leituras.append(leitura)
    return leitura

@app.get("/dispositivos/{dispositivo_id}/leituras", response_model=list[Leitura])
def listar_leituras(dispositivo_id: int):
    if dispositivo_id not in dispositivos:
        raise HTTPException(status_code=404, detail="Dispositivo não encontrado")
    return [l for l in leituras if l.dispositivo_id == dispositivo_id]

```

Executando

Terminal

```

fastapi dev main.py
# alternativa clássica:
# uvicorn main:app --reload

```

A documentação interativa normalmente fica disponível em /docs. Esse recurso é útil para exploração, mas os testes de laboratório também devem ser feitos via linha de comando ou cliente programático para tornar explícita a mensagem HTTP.

Testando com curl

Terminal

```

curl -i http://127.0.0.1:8000/dispositivos

curl -i -X POST http://127.0.0.1:8000/dispositivos \
-H "Content-Type: application/json" \
-d '{"nome":"sensor-sala","local":"Laboratório 1"}'

curl -i -X POST http://127.0.0.1:8000/dispositivos/1/leituras \
-H "Content-Type: application/json" \
-d '{"valor":24.7,"unidade":"C"}'

curl -i http://127.0.0.1:8000/dispositivos/1/leituras

```

Cliente programático

cliente.py

```
import requests

BASE = "http://127.0.0.1:8000"

r = requests.post(
    f'{BASE}/dispositivos',
    json={"nome": "sensor-corredor", "local": "Bloco A"},
    timeout=3,
)
print("POST", r.status_code, r.json())
dispositivo = r.json()

r = requests.get(f'{BASE}/dispositivos/{dispositivo["id"]}', timeout=3)
print("GET", r.status_code, r.json())

try:
    requests.get(f'{BASE}/dispositivos', timeout=0.001)
except requests.exceptions.Timeout:
    print("Timeout observado: a chamada remota não tem duração garantida.")
```

4.5 Stateless, sessão e autenticação

Stateless significa que cada requisição contém as informações necessárias para ser interpretada no contexto da aplicação, em vez de depender de estado de conversação invisível armazenado exclusivamente no servidor. Isso não proíbe banco de dados, cache ou autenticação. Um token pode identificar o usuário em cada requisição; o estado persistente do domínio continua existindo.

4.6 Concorrência e controle de versão

Se dois clientes obtêm a mesma representação e ambos a atualizam, o segundo pode sobrescrever a mudança do primeiro. Em sistemas reais, mecanismos como ETag/If-Match, versionamento otimista ou transações no armazenamento ajudam a detectar concorrência. O problema não é resolvido simplesmente por usar HTTP.

4.7 Observabilidade e erros

Uma API distribuída deve produzir logs estruturados contendo método, caminho, status, duração e identificador de correlação. Evite registrar senhas e tokens. Diferencie erros esperados do domínio de falhas internas. Para depuração distribuída, IDs de correlação permitem seguir uma operação ao atravessar vários serviços.

4.8 Experimentos orientados

- Execute chamadas com servidor desligado e compare `ConnectionError` com resposta HTTP 503.
- Introduza um `sleep` de 2 segundos em uma rota e use `timeouts` de 0,5 s, 1 s e 3 s.
- Faça duas requisições `DELETE` ao mesmo URI e discuta idempotência do efeito versus diferença de status.
- Repita um `POST` de criação e observe por que a operação não é idempotente.
- Adicione validação de unidade permitida e defina código de status coerente para entrada inválida.

AP2 - API REST de gerenciamento de recursos

Atividade prática

Construa uma API REST de um domínio diferente do exemplo da apostila. A atividade deve demonstrar modelagem de recursos, semântica HTTP, validação, tratamento de erros e testes de comportamento.

Objetivos da atividade

- Projetar uma API orientada a recursos.
- Implementar operações CRUD coerentes.
- Testar sucesso, erro, idempotência e timeout.
- Justificar escolhas de URI, método e status HTTP.

Requisitos obrigatórios

- Pelo menos duas coleções relacionadas.
- No mínimo seis endpoints, incluindo GET, POST, PUT ou PATCH e DELETE.
- Validação de entrada e tratamento explícito de 404 e 409 ou erro equivalente.
- Cliente de teste programático com timeout definido.
- Persistência em arquivo ou banco é recomendada, mas a lógica distribuída vale mais que a tecnologia de banco.

Roteiro mínimo

1. Modele recursos e relações antes de programar.
2. Implemente a API e gere dados de teste.
3. Crie roteiro automatizado ou semi-automatizado de testes.
4. Provoque pelo menos dois erros de aplicação e uma falha de conectividade.
5. Colete logs e organize evidências para demonstração.

Entregáveis

- Código-fonte organizado.
- README com instruções de execução.
- Coleção de testes/curl ou script cliente.
- Tabela contendo cenário, requisição, status esperado e status obtido.
- Demonstração prática curta.

Questões para análise

1. Quais operações são idempotentes e por quê?
2. Que diferença existe entre “servidor respondeu erro” e “cliente não conseguiu obter resposta”?
3. Que decisões da sua API aumentam ou reduzem acoplamento entre cliente e servidor?
4. Como evitar perda de atualização quando dois clientes editam o mesmo recurso?

Critérios de avaliação

Tabela 2 - Critérios de avaliação da AP2

Critério	Evidência esperada
Modelagem REST	Recursos e métodos coerentes com o domínio.

Critério	Evidência esperada
Correção técnica	API executa e trata entradas e erros.
Experimentação	Evidências de testes de sucesso e falha.
Análise	Explicações técnicas sobre semântica e distribuição.
Apresentação	Demonstração reproduzível e objetiva.

Checklist antes da apresentação

- ☐ A API inicia a partir de instruções do README.
- ☐ Todos os endpoints obrigatórios foram testados.
- ☐ Existem evidências de pelo menos três cenários de erro.
- ☐ Timeouts são definidos no cliente.
- ☐ Os resultados foram analisados, não apenas capturados.

Desafios opcionais

- Implemente ETag/versionamento otimista.
- Adicione autenticação baseada em token.
- Execute dois clientes concorrentes e observe conflitos.

4.9 Cache, validação condicional e representação

HTTP possui mecanismos de cache que podem reduzir latência e carga. Cache-Control define políticas como max-age; ETag identifica uma versão da representação; If-None-Match permite ao cliente perguntar se houve mudança e receber 304 Not Modified. Esses mecanismos são especialmente úteis para recursos de leitura frequente e alteração pouco frequente. Caches, entretanto, introduzem o risco de dados obsoletos e precisam fazer parte do modelo de consistência percebida.

Content negotiation permite representar o mesmo recurso em formatos diferentes por cabeçalhos como Accept. Na maioria dos laboratórios usaremos JSON por simplicidade, mas APIs públicas podem oferecer múltiplas representações. O recurso é a entidade conceitual; JSON é uma representação dele.

4.10 Paginação, filtros e limites

Listagens sem limite são um erro comum. À medida que o conjunto cresce, uma única requisição pode consumir memória, banda e tempo excessivos. APIs devem definir paginação por página/tamanho, cursor ou token de continuação. Cursor é preferível em conjuntos muito mutáveis porque reduz inconsistências causadas por inserções entre páginas. Também é prudente limitar tamanho máximo de payload e quantidade de itens por requisição.

4.11 Versionamento e compatibilidade

Mudanças compatíveis adicionam campos opcionais ou novos endpoints sem quebrar clientes existentes. Renomear campos, alterar tipos ou mudar semântica pode exigir nova versão. Estratégias incluem versão no caminho (/v1), cabeçalhos ou evolução compatível sem versão explícita. A escolha é menos importante que a disciplina de contrato, depreciação e observação de clientes em uso.

4.12 Segurança prática de APIs

A API deve validar entrada no limite de confiança e não assumir que o cliente respeita a interface. Além de autenticação e autorização, considere rate limiting, proteção contra payloads excessivos, CORS quando browsers participam, logs sem dados sensíveis e mensagens de erro que não revelem detalhes internos. SQL injection é problema da camada de persistência e deve ser prevenido com consultas parametrizadas; não é resolvido por REST.

4.13 Testes por propriedade e contrato

Além de testes endpoint a endpoint, é útil escrever invariantes: depois de DELETE, GET deve retornar ausência; criar recurso inválido nunca deve persistir estado; repetir PUT idêntico não deve alterar o resultado final. Testes de contrato verificam que cliente e servidor concordam sobre esquema e semântica. Em sistemas com várias equipes, esses testes reduzem surpresas de integração.